



Universidad Nacional Autónoma De México
Facultad de Ingeniería
División Ingeniería Eléctrica

Semestre 2026-2 Grupo: 5
Semestre 2026-2

Estructura y Programación de Computadoras

Bitácora IA - Proyecto Final

Profesora: Ing. Ariel Adara Mercado Martinez.

Integrantes:

ESPEJEL ORNELAS IRVIN GIOVANNI
GONZÁLEZ MANCILLA JOSÉ LUIS
RIOS CRUZ RICARDO
SALINAS RUIZ CRISTIAN DANIEL
VILLAVICENCIO ZÁRATE MIGUEL ÁNGEL

Fecha de Realización: 30 de Mayo del 2025
Fecha de Entrega: 11 de Junio del 2026

Integrante 1:

Prompt 1: Estoy programando el analizador léxico en C para un ensamblador. En lugar de guardar todos los tokens del archivo .asm en un arreglo global que podría desbordarse, ¿cómo implemento un patrón de iterador con una función obtener_siguiente_token() que el Parser pueda llamar bajo demanda?

Prompt 2: ¿Cómo estructuro un archivo de cabecera (tokens.h) con un enum que contenga todas las instrucciones de IA-32 y directivas del ensamblador, de manera que pueda agrupar los registros asignándoles un valor numérico inicial (ej. REG_EAX = 250) para facilitar su clasificación lógica posterior?

Prompt 3: En mi estructura Token, tengo el TokenType tipo y un char lexema[64]. Al extraer las cadenas del archivo fuente carácter por carácter, ¿cómo me aseguro de limpiar los saltos de línea huérfanos, ignorar los espacios y aislar correctamente las comas de puntuación antes de copiar la cadena al campo lexema?

Prompt 4: El ensamblador es estricto para la arquitectura IA-32. ¿Cuál es la forma más eficiente en C de validar que los operandos sean únicamente registros de 32 bits (EAX, EBX, ECX, etc.), exigiendo la letra 'E' al inicio y rechazando las variantes de 16 bits, sin hacer una cadena interminable de if-else?

Prompt 5: Desarrollé un archivo de pruebas de estrés con sintaxis inválida y caracteres especiales (como @). ¿Cómo estructurar correctamente la salida de error en mi clasificador léxico para que imprima '[ADVERTENCIA LÉXICA]' en la salida estándar de errores (stderr) y emita un TOKEN_DESCONOCIDO sin que el programa en C colapse por un Segmentation Fault?

Código generado:

tokens.h:

```
#ifndef TOKENS_H
#define TOKENS_H
```

```
// Definición del diccionario de tokens (Arquitectura estricta IA-32)
```

```
typedef enum {
```

```
    REG_EAX = 250, REG_EBX, REG_ECX, REG_EDX,
    REG_ESI, REG_EDI, REG_EBP, REG_ESP,
```

```
    INST_MOV, INST_PUSH, INST_POP, INST_LEA, INST_MOVZX,
    INST_ADD, INST_SUB, INST_INC, INST_DEC, INST_MUL, INST_DIV, INST_IMUL,
    INST_IDIV, INST_CMP, INST_NEG,
    INST_AND, INST_OR, INST_XOR, INST_NOT,
```



INST_JMP, INST_JE, INST_JNE, INST_JG, INST_JGE, INST_JL, INST_JLE,
INST_CALL, INST_RET, INST_NOP, INST_INT,

DIR_DB, DIR_DW, DIR_DD, DIR_RESB, DIR_RESW, DIR_RESD,
DIR_SECTION, DIR_GLOBAL, DIR_EXTERN, DIR_EQU,

TOKEN_INMEDIATO,
TOKEN_ETIQUETA,
TOKEN_IDENTIFICADOR,
TOKEN_EOF,
TOKEN_DESCONOCIDO
} TokenType;

```
typedef struct {  
    TokenType tipo;  
    char lexema[64];  
} Token;
```

```
// Prototipos (Implementación del Patrón Iterador para evitar desbordamientos)  
void iniciar_lexer(const char *filename);  
Token obtener_siguiente_token();  
void cerrar_lexer();
```

#endif

Lexer.c:

```
#include <stdio.h>  
#include <string.h>  
#include <ctype.h>  
#include "tokens.h"
```

```
typedef struct {  
    const char *palabra;  
    TokenType tipo;  
} DiccionarioToken;
```

```
// Tabla estática para validar eficientemente palabras reservadas e ignorar 16 bits  
static const DiccionarioToken tabla_busqueda[] = {  
    {"EAX", REG_EAX}, {"EBX", REG_EBX}, {"ECX", REG_ECX}, {"EDX", REG_EDX},  
    {"ESI", REG_ESI}, {"EDI", REG_EDI}, {"EBP", REG_EBP}, {"ESP", REG_ESP},  
    {"MOV", INST_MOV}, {"PUSH", INST_PUSH}, {"POP", INST_POP}, {"LEA", INST_LEA},  
    {"ADD", INST_ADD}, {"SUB", INST_SUB}, {"INC", INST_INC}, {"DEC", INST_DEC},  
    {"JMP", INST_JMP}, {"JE", INST_JE}, {"JNE", INST_JNE}, {"NOP", INST_NOP}, {"INT",  
INST_INT},  
    {"DB", DIR_DB}, {"DW", DIR_DW}, {"DD", DIR_DD},  
    {"SECTION", DIR_SECTION}, {"GLOBAL", DIR_GLOBAL}, {"EXTERN", DIR_EXTERN}  
};
```



UNAM



```
static const int num_palabras = sizeof(tabla_busqueda) / sizeof(tabla_busqueda[0]);
```

```
static FILE *archivo_fuente = NULL;
```

```
static char linea_actual[256];
```

```
static char *token_actual = NULL;
```

```
void iniciar_lexer(const char *filename) {  
    archivo_fuente = fopen(filename, "r");  
    if (!archivo_fuente) {  
        printf("Error al abrir el archivo %s\n", filename);  
    }  
}
```

```
void cerrar_lexer() {  
    if (archivo_fuente) {  
        fclose(archivo_fuente);  
        archivo_fuente = NULL;  
    }  
}
```

```
// Tolerancia a mayúsculas/minúsculas
```

```
void estandarizar_token(char *str) {  
    for (int i = 0; str[i]; i++) {  
        str[i] = toupper((unsigned char)str[i]);  
    }  
}
```

```
TokenType clasificar_lexema(const char *lexema) {  
    int len = strlen(lexema);  
    if (len > 0 && lexema[len - 1] == ':') return TOKEN_ETIQUETA;
```

```
    for (int i = 0; i < num_palabras; i++) {  
        if (strcmp(lexema, tabla_busqueda[i].palabra) == 0) {  
            return tabla_busqueda[i].tipo;  
        }  
    }  
}
```

```
// Si empieza con comillas, es una cadena de texto (lo tratamos como inmediato)
```

```
if (lexema[0] == "\"" || lexema[0] == "'") {  
    return TOKEN_INMEDIATO;  
}
```

```
// Si es un número (positivo o negativo)
```

```
if (isdigit(lexema[0]) || (lexema[0] == '-' && isdigit(lexema[1]))) {  
    return TOKEN_INMEDIATO;  
}
```



```
// Si empieza con letra, guion bajo, o un PUNTO (para que pase .DATA y .TEXT)
if (isalpha(lexema[0]) || lexema[0] == '_' || lexema[0] == '.') {
    return TOKEN_IDENTIFICADOR;
}

// MANEJO BÁSICO DE ERRORES LÉXICOS (Emisión de error seguro)
printf(">> [ADVERTENCIA LEXICA] Simbolo no reconocido: '%s'\n", lexema);
return TOKEN_DESCONOCIDO;
}

// Función principal del Iterador
Token obtener_siguiente_token() {
    Token t;
    t.tipo = TOKEN_EOF;
    t.lexema[0] = '\0';

    if (archivo_fuente == NULL) return t;

    // Si no hay token actual o ya se consumió la línea, leemos la siguiente
    while (token_actual == NULL) {
        if (fgets(linea_actual, sizeof(linea_actual), archivo_fuente) == NULL) return t;

        // Truncar comentarios inmediatamente
        char *comentario = strchr(linea_actual, ';');
        if (comentario != NULL) *comentario = '\0';

        // Aislar los componentes con los delimitadores de IA-32, comas y espacios
        token_actual = strtok(linea_actual, " ,\n\r\t[+*");
    }

    strcpy(t.lexema, token_actual);
    estandarizar_token(t.lexema);
    t.tipo = clasificar_lexema(t.lexema);

    // Avanzar el puntero interno de strtok para la próxima llamada
    token_actual = strtok(NULL, " ,\n\r\t[+*");
    return t;
}
```

Integrante 2:

Prompts para el archivo parser.h

Prompt 1: Diseño de la estructura del operando con soporte SIB

Estoy diseñando la estructura AST en C para un ensamblador IA-32. Necesito crear un struct Operand que soporte direccionamiento de memoria, específicamente el formato SIB (Base, Índice, Escala, Desplazamiento), además de registros e inmediatos. ¿Cómo puedo



estructurarlo de forma eficiente usando arrays estáticos en lugar de memoria dinámica? Solo entrégame el código de la estructura en C. No agregues comentarios en el programa o emojis.

Prompt 2: Anidación de estructuras para la instrucción

Ya tengo definido el struct Operand. Ahora requiero construir el struct Instruction principal. Este debe almacenar el mnemónico de la operación (máximo 10 caracteres) y debe ser capaz de contener hasta dos operandos, además de un contador que indique cuántos operandos se leyeron. Genera esta estructura de C. No quiero comentarios en el programa o emojis.

Prompt 3: Definición de la interfaz con el Lexer

Para que mi parser pueda comunicarse con el código del Integrante 1 (Lexer), necesito definir en mi cabecera los tipos de datos de entrada. Genera un enum TokenType con los elementos básicos (identificador, número, coma, corchetes, operadores, salto de línea) y un struct Token que contenga el tipo y un valor de texto de 32 bytes. Entrega solo las definiciones. No agregues comentarios en el programa o emojis.

Prompts para el archivo parser.c

Prompt 4: Manejo de punteros para el recorrido de tokens

Estoy implementando el parser manual por descenso recursivo en C. Voy a recibir un arreglo de tokens desde el Lexer. ¿Cómo debo declarar una función de avance `advance(Token** current)` que reciba un puntero doble para consumir el token actual y mover el puntero global al siguiente elemento del arreglo? Escribe solo la función. No quiero comentarios en el programa o emojis.

Prompt 5: Algoritmo de extracción para direccionamiento de memoria

Necesito la lógica interna para la función `parse_memory_operand`. El algoritmo debe iterar dentro de unos corchetes [...] y clasificar los tokens: el primer registro es la base, el segundo es el índice, si hay un asterisco seguido de un número es la escala, y los números sueltos se suman al desplazamiento. Escribe el ciclo while que procesa esto usando los tokens. No agregues comentarios en el programa o emojis.

Prompt 6: Validación sintáctica de separación por comas

En mi función `parse_instruction`, ya logro extraer el mnemónico y el primer operando. Necesito implementar la validación sintáctica estricta para el segundo operando: si el siguiente token no es un salto de línea o EOF, obligatoriamente debe ser una coma ,. Si falta la coma, debe imprimir un error y hacer `exit(1)`. Dame el fragmento condicional en C que resuelve esto. No quiero comentarios en el programa o emojis.

Códigos finales:

```
#ifndef PARSE_H
#define PARSE_H
```

```
#include <stdint.h>
#include <stdbool.h>
```



UNAM

```
typedef enum {
    TOK_IDENTIFIER,
    TOK_NUMBER,
    TOK_COMMA,
    TOK_LBRACKET,
    TOK_RBRACKET,
    TOK_PLUS,
    TOK_STAR,
    TOK_NEWLINE,
    TOK_EOF
} TokenType;

typedef struct {
    TokenType type;
    char value[32];
} Token;

typedef enum {
    OP_NONE,
    OP_REG,
    OP_IMM,
    OP_MEM
} OperandType;

typedef struct {
    OperandType type;
    char reg_name[4];
    int32_t imm_value;
    char base_reg[4];
    char index_reg[4];
    int scale;
    int32_t displacement;
    bool has_base;
    bool has_index;
} Operand;

typedef struct {
    char mnemonic[10];
    Operand op1;
    Operand op2;
    int operand_count;
} Instruction;

typedef struct {
    char name[16];
    char arguments[4][32];
    int arg_count;
```





```
} Directive;
```

```
Operand parse_operand(Token** current_token);  
Operand parse_memory_operand(Token** current_token);  
Instruction parse_instruction(Token** current_token);  
Directive parse_directive(Token** current_token);
```

```
#endif
```

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include "parser.h"
```

```
void advance(Token** current) {  
    (*current)++;  
}
```

```
bool is_register(const char* str) {  
    const char* regs[] = {"EAX", "EBX", "ECX", "EDX", "ESI", "EDI", "EBP", "ESP"};  
    for (int i = 0; i < 8; i++) {  
        if (strcmp(str, regs[i]) == 0) return true;  
    }  
    return false;  
}
```

```
Operand parse_operand(Token** current) {  
    Operand op;  
    memset(&op, 0, sizeof(Operand));  
    op.type = OP_NONE;  
    op.scale = 1;
```

```
    Token* tok = *current;
```

```
    if (tok->type == TOK_NUMBER) {  
        op.type = OP_IMM;  
        op.imm_value = atoi(tok->value);  
        advance(current);  
        return op;  
    }
```

```
    if (tok->type == TOK_IDENTIFIER && is_register(tok->value)) {  
        op.type = OP_REG;  
        strcpy(op.reg_name, tok->value);  
        advance(current);  
        return op;  
    }
```




```
if (tok->type == TOK_LBRACKET) {
    return parse_memory_operand(current);
}

if (tok->type == TOK_IDENTIFIER) {
    op.type = OP_IMM;
    strcpy(op.reg_name, tok->value);
    advance(current);
    return op;
}

printf("Error de sintaxis: Operando no reconocido \"%s\\n", tok->value);
exit(1);
}

Operand parse_memory_operand(Token** current) {
    Operand mem_op;
    memset(&mem_op, 0, sizeof(Operand));
    mem_op.type = OP_MEM;
    mem_op.scale = 1;

    advance(current);

    while ((*current)->type != TOK_RBRACKET && (*current)->type != TOK_EOF) {
        Token* tok = *current;

        if (tok->type == TOK_IDENTIFIER && is_register(tok->value)) {
            if (!mem_op.has_base) {
                strcpy(mem_op.base_reg, tok->value);
                mem_op.has_base = true;
                advance(current);
            } else if (!mem_op.has_index) {
                strcpy(mem_op.index_reg, tok->value);
                mem_op.has_index = true;
                advance(current);
            }

            if ((*current)->type == TOK_STAR) {
                advance(current);
                if ((*current)->type == TOK_NUMBER) {
                    mem_op.scale = atoi((*current)->value);
                    advance(current);
                }
            }
        }
    }

    else if (tok->type == TOK_PLUS) {
        advance(current);
    }
}
```



```
else if (tok->type == TOK_NUMBER) {
    mem_op.displacement += atoi(tok->value);
    advance(current);
} else {
    advance(current);
}
}

if ((*current)->type == TOK_RBRACKET) {
    advance(current);
} else {
    printf("Error: Se esperaba '']'\n");
    exit(1);
}

return mem_op;
}

Instruction parse_instruction(Token** current) {
    Instruction inst;
    memset(&inst, 0, sizeof(Instruction));

    strncpy(inst.mnemonic, (*current)->value, sizeof(inst.mnemonic) - 1);
    advance(current);

    if ((*current)->type != TOK_NEWLINE && (*current)->type != TOK_EOF) {
        inst.op1 = parse_operand(current);
        inst.operand_count = 1;

        if ((*current)->type == TOK_COMMA) {
            advance(current);
            inst.op2 = parse_operand(current);
            inst.operand_count = 2;
        } else if ((*current)->type != TOK_NEWLINE && (*current)->type != TOK_EOF) {
            printf("Error de sintaxis: Se esperaba una coma ',' despues del operando.\n");
            exit(1);
        }
    }

    return inst;
}

Directive parse_directive(Token** current) {
    Directive dir;
    memset(&dir, 0, sizeof(Directive));

    strncpy(dir.name, (*current)->value, sizeof(dir.name) - 1);
    advance(current);
}
```

```
while ((*current)->type != TOK_NEWLINE && (*current)->type != TOK_EOF) {
    if ((*current)->type == TOK_COMMA) {
        advance(current);
        continue;
    }

    if (dir.arg_count < 4) {
        strncpy(dir.arguments[dir.arg_count], (*current)->value, 31);
        dir.arg_count++;
    }
    advance(current);
}

return dir;
}
```

Integrante 3:

Prompt 1 (Análisis y Estructuración): *"Actúa como un ingeniero de software experto en sistemas embebidos y arquitectura de computadoras IA-32. Con base en el PDF de requerimientos del proyecto final, necesito diseñar el entorno del Integrante 3 (Backend del Ensamblador) en lenguaje C puro sin usar bibliotecas automáticas. Provee las estructuras de datos esenciales para una Tabla de Símbolos, una Tabla de Fixups para referencias adelantadas y las firmas de funciones necesarias orientadas a una arquitectura homogénea y modular."*

Prompt 2 (Acoplamiento con el Frontend): *"Tengo este archivo parser.c y su cabecera parser.h desarrollados por el Integrante 2 de mi equipo. Necesito que crees la lógica de la Primera Pasada (ejecutar_primera_pasada) en C que recorra el flujo de tokens provisto por el lexer/parser, controle el avance del Location Counter, gestione los cambios de sección con SECTION .data o .text, e intercepte las etiquetas de saltos (JMP/CALL) para meterlas en la tabla de fixups si son referencias adelantadas."*

Prompt 3 (Refactorización y Corrección IA-32): *"Aquí está mi código actual de backend.c. Optimízalo e integra una función para alternar entre los Location Counters de las secciones .text, .data y .bss de forma independiente. Además, corrige la función de resolución de fixups (fixup_resolve_all): recuerda que en la arquitectura IA-32, los opcodes de salto en la sección .text requieren un desplazamiento relativo calculado como destino - (posición_parche + tamaño_parche) en formato Little Endian, mientras que las directivas de datos usan direcciones absolutas. Entrégame el archivo completo unificado."*

Código generado:



UNAM



```
#include "../include/backend.h"
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

// Definición de variables globales
Symbol symbol_table[MAX_SYMBOLS];
int symbol_count = 0;

Fixup fixup_table[MAX_FIXUPS];
int fixup_count = 0;

uint32_t location_counter = 0;
char current_section[10] = ".text"; // Sección por defecto obligatoria

// Control de Location Counters por sección
uint32_t section_lc[3] = {0, 0, 0};
int indice_seccion_actual = 0; // 0 = .text, 1 = .data, 2 = .bss

// Busca un símbolo en la tabla. Devuelve el índice o -1 si no existe.
int symbol_lookup(const char* name) {
    for (int i = 0; i < symbol_count; i++) {
        if (strcmp(symbol_table[i].name, name) == 0) {
            return i;
        }
    }
    return -1;
}

// Inserta un nuevo símbolo. Retorna false si hay redefinición (Manejo de Errores).
bool symbol_insert(const char* name, uint32_t address, const char* section, bool is_extern,
bool is_global) {
    int idx = symbol_lookup(name);
    if (idx != -1) {
        printf("Error Semantico: El simbolo '%s' ya fue definido.\n", name);
        return false;
    }

    if (symbol_count >= MAX_SYMBOLS) {
        printf("Error: Tabla de simbolos llena.\n");
        return false;
    }

    strcpy(symbol_table[symbol_count].name, name);
    symbol_table[symbol_count].address = address;
    strcpy(symbol_table[symbol_count].section, section);
    symbol_table[symbol_count].is_extern = is_extern;
```



```
symbol_table[symbol_count].is_global = is_global;

symbol_count++;
return true;
}

// Registra una referencia adelantada para parcharla posteriormente
void fixup_add(const char* symbol_name, uint32_t patch_pos, uint32_t size, const char*
section) {
    if (fixup_count >= MAX_FIXUPS) {
        printf("Error: Tabla de fixups llena.\n");
        return;
    }

    strcpy(fixup_table[fixup_count].symbol_name, symbol_name);
    fixup_table[fixup_count].patch_pos = patch_pos;
    fixup_table[fixup_count].size = size;
    strcpy(fixup_table[fixup_count].section, section);

    fixup_count++;
}

// Resuelve todos los parches pendientes modificando el buffer de bytes del Encoder (IA-32)
void fixup_resolve_all(uint8_t* code_buffer) {
    for (int i = 0; i < fixup_count; i++) {
        int sym_idx = symbol_lookup(fixup_table[i].symbol_name);
        if (sym_idx == -1) {
            printf("Error de Enlazado/Ensamblado: Simbolo '%s' no resuelto.\n",
fixup_table[i].symbol_name);
            continue;
        }

        uint32_t target_address = symbol_table[sym_idx].address;
        uint32_t patch_pos = fixup_table[i].patch_pos;
        uint32_t valor_a_inyectar = target_address;

        // IA-32: Los saltos en la sección ejecutable (.text) son relativos al PC actual
        if (strcmp(fixup_table[i].section, ".text") == 0) {
            valor_a_inyectar = target_address - (patch_pos + fixup_table[i].size);
        }

        // Inyección Little Endian
        if (fixup_table[i].size == 1) {
            code_buffer[patch_pos] = (uint8_t)(valor_a_inyectar & 0xFF);
        } else if (fixup_table[i].size == 4) {
            code_buffer[patch_pos] = (uint8_t)(valor_a_inyectar & 0xFF);
            code_buffer[patch_pos + 1] = (uint8_t)((valor_a_inyectar >> 8) & 0xFF);
            code_buffer[patch_pos + 2] = (uint8_t)((valor_a_inyectar >> 16) & 0xFF);
        }
    }
}
```



```
code_buffer[patch_pos + 3] = (uint8_t)((valor_a_inyectar >> 24) & 0xFF);
}
printf("Fixup aplicado con exito: Simbolo '%s' en pos %u con valor %d\n",
      fixup_table[i].symbol_name, patch_pos, (int32_t)valor_a_inyectar);
}
}

// Cambia y segmenta el Location Counter por secciones (.text, .data, .bss)
void cambiar_de_seccion(const char* nueva_seccion) {
    section_lc[indice_seccion_actual] = location_counter;
    strncpy(current_section, nueva_seccion, sizeof(current_section) - 1);

    if (strcmp(nueva_seccion, ".text") == 0) {
        indice_seccion_actual = 0;
    } else if (strcmp(nueva_seccion, ".data") == 0) {
        indice_seccion_actual = 1;
    } else if (strcmp(nueva_seccion, ".bss") == 0) {
        indice_seccion_actual = 2;
    } else {
        printf("Error: Seccion '%s' no reconocida.\n", nueva_seccion);
        return;
    }

    location_counter = section_lc[indice_seccion_actual];
    printf("[BACKEND] Cambiando a seccion %s. LC restaurado en: %u\n", current_section,
location_counter);
}

// Estima provisionalmente el peso en bytes de la instrucción (Pasada 1)
uint32_t estimate_instruction_size(const char* mnemonic, int op_count) {
    if (strcmp(mnemonic, "NOP") == 0 || strcmp(mnemonic, "RET") == 0) {
        return 1;
    }
    if (strcmp(mnemonic, "JMP") == 0 || mnemonic[0] == 'J') {
        return 5; // Opcode + 4 bytes de offset relativo largo
    }
    if (strcmp(mnemonic, "MOV") == 0) {
        if (op_count == 2) return 5;
    }
    return 3; // Resguardo genérico
}
```

2. Lógica Central

```
#include "../include/backend.h"
#include <string.h>
#include <stdio.h>
```



#include <stdlib.h>

// Definición de variables globales

Symbol symbol_table[MAX_SYMBOLS];

int symbol_count = 0;

Fixup fixup_table[MAX_FIXUPS];

int fixup_count = 0;

uint32_t location_counter = 0;

char current_section[10] = ".text"; // Sección por defecto obligatoria

// Control de Location Counters por sección

uint32_t section_lc[3] = {0, 0, 0};

int indice_seccion_actual = 0; // 0 = .text, 1 = .data, 2 = .bss

// Busca un símbolo en la tabla. Devuelve el índice o -1 si no existe.

```
int symbol_lookup(const char* name) {
    for (int i = 0; i < symbol_count; i++) {
        if (strcmp(symbol_table[i].name, name) == 0) {
            return i;
        }
    }
    return -1;
}
```

// Inserta un nuevo símbolo. Retorna false si hay redefinición (Manejo de Errores).

bool symbol_insert(const char* name, uint32_t address, const char* section, bool is_extern, bool is_global) {

```
    int idx = symbol_lookup(name);
    if (idx != -1) {
        printf("Error Semantico: El simbolo '%s' ya fue definido.\n", name);
        return false;
    }
```

```
    if (symbol_count >= MAX_SYMBOLS) {
        printf("Error: Tabla de simbolos llena.\n");
        return false;
    }
```

```
    strcpy(symbol_table[symbol_count].name, name);
    symbol_table[symbol_count].address = address;
    strcpy(symbol_table[symbol_count].section, section);
    symbol_table[symbol_count].is_extern = is_extern;
    symbol_table[symbol_count].is_global = is_global;
```

```
    symbol_count++;
    return true;
```





```
// Registra una referencia adelantada para parcharla posteriormente
void fixup_add(const char* symbol_name, uint32_t patch_pos, uint32_t size, const char*
section) {
    if (fixup_count >= MAX_FIXUPS) {
        printf("Error: Tabla de fixups llena.\n");
        return;
    }

    strcpy(fixup_table[fixup_count].symbol_name, symbol_name);
    fixup_table[fixup_count].patch_pos = patch_pos;
    fixup_table[fixup_count].size = size;
    strcpy(fixup_table[fixup_count].section, section);

    fixup_count++;
}

// Resuelve todos los parches pendientes modificando el buffer de bytes del Encoder (IA-32)
void fixup_resolve_all(uint8_t* code_buffer) {
    for (int i = 0; i < fixup_count; i++) {
        int sym_idx = symbol_lookup(fixup_table[i].symbol_name);
        if (sym_idx == -1) {
            printf("Error de Enlazado/Ensamblado: Simbolo '%s' no resuelto.\n",
fixup_table[i].symbol_name);
            continue;
        }

        uint32_t target_address = symbol_table[sym_idx].address;
        uint32_t patch_pos = fixup_table[i].patch_pos;
        uint32_t valor_a_inyectar = target_address;

        // IA-32: Los saltos en la sección ejecutable (.text) son relativos al PC actual
        if (strcmp(fixup_table[i].section, ".text") == 0) {
            valor_a_inyectar = target_address - (patch_pos + fixup_table[i].size);
        }

        // Inyección Little Endian
        if (fixup_table[i].size == 1) {
            code_buffer[patch_pos] = (uint8_t)(valor_a_inyectar & 0xFF);
        } else if (fixup_table[i].size == 4) {
            code_buffer[patch_pos] = (uint8_t)(valor_a_inyectar & 0xFF);
            code_buffer[patch_pos + 1] = (uint8_t)((valor_a_inyectar >> 8) & 0xFF);
            code_buffer[patch_pos + 2] = (uint8_t)((valor_a_inyectar >> 16) & 0xFF);
            code_buffer[patch_pos + 3] = (uint8_t)((valor_a_inyectar >> 24) & 0xFF);
        }

        printf("Fixup aplicado con exito: Simbolo '%s' en pos %u con valor %d\n",
            fixup_table[i].symbol_name, patch_pos, (int32_t)valor_a_inyectar);
    }
}
```




```
// Cambia y segmenta el Location Counter por secciones (.text, .data, .bss)
void cambiar_de_seccion(const char* nueva_seccion) {
    section_lc[indice_seccion_actual] = location_counter;
    strncpy(current_section, nueva_seccion, sizeof(current_section) - 1);

    if (strcmp(nueva_seccion, ".text") == 0) {
        indice_seccion_actual = 0;
    } else if (strcmp(nueva_seccion, ".data") == 0) {
        indice_seccion_actual = 1;
    } else if (strcmp(nueva_seccion, ".bss") == 0) {
        indice_seccion_actual = 2;
    } else {
        printf("Error: Seccion '%s' no reconocida.\n", nueva_seccion);
        return;
    }

    location_counter = section_lc[indice_seccion_actual];
    printf("[BACKEND] Cambiando a seccion %s. LC restaurado en: %u\n", current_section,
location_counter);
}

// Estima provisionalmente el peso en bytes de la instrucción (Pasada 1)
uint32_t estimate_instruction_size(const char* mnemonic, int op_count) {
    if (strcmp(mnemonic, "NOP") == 0 || strcmp(mnemonic, "RET") == 0) {
        return 1;
    }
    if (strcmp(mnemonic, "JMP") == 0 || mnemonic[0] == 'J') {
        return 5; // Opcode + 4 bytes de offset relativo largo
    }
    if (strcmp(mnemonic, "MOV") == 0) {
        if (op_count == 2) return 5;
    }
    return 3; // Resguardo genérico
}
```

Proceso Integrado:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "../include/parser.h"
#include "../include/backend.h"

void ejecutar_primera_pasada(Token* token_list) {
    Token* current = token_list;
```



```
location_counter = 0;  
cambiar_de_seccion(".text");
```

```
while (current->type != TOK_EOF) {  
    if (current->type == TOK_NEWLINE) {  
        advance(&current);  
        continue;  
    }  
  
    // --- 1. DETECCIÓN DE ETIQUETAS (Ej: inicio:) ---  
    if (current->type == TOK_IDENTIFIER && strcmp((current + 1)->value, ":") == 0) {  
        bool exito = symbol_insert(current->value, location_counter, current_section, false,  
false);  
        if (!exito) {  
            printf("Compilacion abortada por errores semanticos.\n");  
            exit(1);  
        }  
        advance(&current); // Consumir identificador  
        advance(&current); // Consumir ':'  
        continue;  
    }  
  
    // --- 2. DETECCIÓN DE DIRECTIVAS (.data, DB, DD, EXTERN...) ---  
    if (current->type == TOK_IDENTIFIER &&  
        (strcmp(current->value, "SECTION") == 0 || strcmp(current->value, "DB") == 0 ||  
         strcmp(current->value, "DW") == 0 || strcmp(current->value, "DD") == 0 ||  
         strcmp(current->value, "GLOBAL") == 0 || strcmp(current->value, "EXTERN") == 0))  
    {  
  
        Directive dir = parse_directive(&current);  
  
        if (strcmp(dir.name, "SECTION") == 0) {  
            cambiar_de_seccion(dir.arguments[0]);  
        }  
        else if (strcmp(dir.name, "DB") == 0) {  
            location_counter += (1 * dir.arg_count);  
        }  
        else if (strcmp(dir.name, "DW") == 0) {  
            location_counter += (2 * dir.arg_count);  
        }  
        else if (strcmp(dir.name, "DD") == 0) {  
            location_counter += (4 * dir.arg_count);  
        }  
        else if (strcmp(dir.name, "GLOBAL") == 0) {  
            int idx = symbol_lookup(dir.arguments[0]);  
            if (idx != -1) symbol_table[idx].is_global = true;  
        }  
        else if (strcmp(dir.name, "EXTERN") == 0) {
```



```
symbol_insert(dir.arguments[0], 0, "extern", true, false);
}

if (current->type == TOK_NEWLINE) advance(&current);
continue;
}

// --- 3. DETECCIÓN DE INSTRUCCIONES INTERCEPTANDO SALTOS
ADELANTADOS ---
if (current->type == TOK_IDENTIFIER) {
    Token* pseudo_check = current;

    if (strcmp(pseudo_check->value, "JMP") == 0 || pseudo_check->value[0] == 'J' ||
        strcmp(pseudo_check->value, "CALL") == 0) {

        char mnemonic[10];
        strcpy(mnemonic, pseudo_check->value);
        advance(&pseudo_check);

        if (pseudo_check->type == TOK_IDENTIFIER &&
            !is_register(pseudo_check->value)) {
            char target_symbol[32];
            strcpy(target_symbol, pseudo_check->value);

            int sym_idx = symbol_lookup(target_symbol);
            if (sym_idx == -1) {
                // FIXUP: Referencia adelantada detectada. Parcharemos en
                (location_counter + 1)
                fixup_add(target_symbol, location_counter + 1, 4, current_section);
            }

            location_counter += estimate_instruction_size(mnemonic, 1);
            current = pseudo_check;
            advance(&current); // Consumir etiqueta destino
            if (current->type == TOK_NEWLINE) advance(&current);
            continue;
        }
    }

    // Instrucción genérica procesada con el Parser del Integrante 2
    Instruction inst = parse_instruction(&current);
    location_counter += estimate_instruction_size(inst.mnemonic, inst.operand_count);

    if (current->type == TOK_NEWLINE) advance(&current);
    continue;
}

advance(&current);
```



Integrante 4:

Prompt: Considerando los archivos de los integrantes 2 y 3 y tomando como base mi estructura para la función, construye la función para el encoder.

Resultado:

```
/* NOP — 90 */
if (strcmp(inst->mnemonic, "NOP") == 0)
{
    output_buffer[0] = 0x90;
    return 1;
}

/* RET — C3 */
if (strcmp(inst->mnemonic, "RET") == 0)
{
    output_buffer[0] = 0xC3;
    return 1;
}

/* PUSH reg — 50+r */
if (strcmp(inst->mnemonic, "PUSH") == 0)
{
    if (inst->operand_count == 1 && inst->op1.type == OP_REG)
    {
        output_buffer[0] = 0x50 + reg_code(inst->op1.reg_name);
        return 1;
    }
}

/* POP reg — 58+r */
if (strcmp(inst->mnemonic, "POP") == 0)
{
    if (inst->operand_count == 1 && inst->op1.type == OP_REG)
    {
        output_buffer[0] = 0x58 + reg_code(inst->op1.reg_name);
        return 1;
    }
}

/* MOV reg, imm32 — B8+r id */
if (strcmp(inst->mnemonic, "MOV") == 0 &&
    inst->operand_count == 2 &&
```

```

inst->op1.type == OP_REG &&
inst->op2.type == OP_IMM)
{
    output_buffer[0] = 0xB8 + reg_code(inst->op1.reg_name);
    write_imm32(&output_buffer[1], inst->op2.imm_value);
    return 5;
}

/* MOV reg, reg — 89 /r */
if (strcmp(inst->mnemonic, "MOV") == 0 &&
    inst->operand_count == 2 &&
    inst->op1.type == OP_REG &&
    inst->op2.type == OP_REG)
{
    int dst = reg_code(inst->op1.reg_name);
    int src = reg_code(inst->op2.reg_name);
    output_buffer[0] = 0x89;
    output_buffer[1] = 0xC0 | (src << 3) | dst;
    return 2;
}

/* MOV reg, [mem+disp] — 8B /r disp32 */
if (strcmp(inst->mnemonic, "MOV") == 0 &&
    inst->op1.type == OP_REG &&
    inst->op2.type == OP_MEM)
{
    int dst = reg_code(inst->op1.reg_name);
    int pos = 0;
    output_buffer[pos++] = 0x8B;
    if (inst->op2.has_base && !inst->op2.has_index)
    {
        int base = reg_code(inst->op2.base_reg);
        output_buffer[pos++] = make_modrm(2, dst, base);
        write_disp32(&output_buffer[pos], inst->op2.displacement);
        pos += 4;
        return pos;
    }
}

/* MOV [mem+disp], reg — 89 /r disp32 */
if (strcmp(inst->mnemonic, "MOV") == 0 &&
    inst->op1.type == OP_MEM &&
    inst->op2.type == OP_REG)
{
    int src = reg_code(inst->op2.reg_name);
    int pos = 0;
    output_buffer[pos++] = 0x89;
    if (inst->op1.has_base && !inst->op1.has_index)

```

```
{
    int base = reg_code(inst->op1.base_reg);
    output_buffer[pos++] = make_modrm(2, src, base);
    write_disp32(&output_buffer[pos], inst->op1.displacement);
    pos += 4;
    return pos;
}
}
```

```
/* ADD reg, reg — 01 /r */
if (strcmp(inst->mnemonic, "ADD") == 0 &&
    inst->op1.type == OP_REG &&
    inst->op2.type == OP_REG)
{
    int dst = reg_code(inst->op1.reg_name);
    int src = reg_code(inst->op2.reg_name);
    output_buffer[0] = 0x01;
    output_buffer[1] = make_modrm(3, src, dst);
    return 2;
}
```

```
/* SUB reg, reg — 29 /r */
if (strcmp(inst->mnemonic, "SUB") == 0 &&
    inst->op1.type == OP_REG &&
    inst->op2.type == OP_REG)
{
    int dst = reg_code(inst->op1.reg_name);
    int src = reg_code(inst->op2.reg_name);
    output_buffer[0] = 0x29;
    output_buffer[1] = make_modrm(3, src, dst);
    return 2;
}
```

```
/* CMP reg, reg — 39 /r */
if (strcmp(inst->mnemonic, "CMP") == 0 &&
    inst->op1.type == OP_REG &&
    inst->op2.type == OP_REG)
{
    int dst = reg_code(inst->op1.reg_name);
    int src = reg_code(inst->op2.reg_name);
    output_buffer[0] = 0x39;
    output_buffer[1] = make_modrm(3, src, dst);
    return 2;
}
```

```
/* JMP rel32 — E9 cd */
if (strcmp(inst->mnemonic, "JMP") == 0)
{
```

```
    output_buffer[0] = 0xE9;
    write_imm32(&output_buffer[1], inst->op1.imm_value);
    return 5;
}

/* CALL rel32 — E8 cd */
if (strcmp(inst->mnemonic, "CALL") == 0)
{
    output_buffer[0] = 0xE8;
    write_imm32(&output_buffer[1], inst->op1.imm_value);
    return 5;
}

/* JE rel32 — 0F 84 cd */
if (strcmp(inst->mnemonic, "JE") == 0)
{
    output_buffer[0] = 0x0F;
    output_buffer[1] = 0x84;
    write_imm32(&output_buffer[2], inst->op1.imm_value);
    return 6;
}

/* JNE rel32 — 0F 85 cd */
if (strcmp(inst->mnemonic, "JNE") == 0)
{
    output_buffer[0] = 0x0F;
    output_buffer[1] = 0x85;
    write_imm32(&output_buffer[2], inst->op1.imm_value);
    return 6;
}

/* INT imm8 — CD ib (ej: INT 0x80) */
if (strcmp(inst->mnemonic, "INT") == 0 &&
    inst->operand_count == 1 &&
    inst->op1.type == OP_IMM)
{
    output_buffer[0] = 0xCD;
    output_buffer[1] = (uint8_t)(inst->op1.imm_value & 0xFF);
    return 2;
}

/* INC reg — 40+r */
if (strcmp(inst->mnemonic, "INC") == 0 &&
    inst->operand_count == 1 &&
    inst->op1.type == OP_REG)
{
    output_buffer[0] = 0x40 + reg_code(inst->op1.reg_name);
    return 1;
}
```



```
}

/* DEC reg — 48+r */
if (strcmp(inst->mnemonic, "DEC") == 0 &&
    inst->operand_count == 1 &&
    inst->op1.type == OP_REG)
{
    output_buffer[0] = 0x48 + reg_code(inst->op1.reg_name);
    return 1;
}

printf("[ENCODER] Instruccion no soportada: %s\n", inst->mnemonic);
return 0;
```

Prompt: Dame la instrucción para convertir los registros a los valores del procesador.

Resultado:

```
if (strcmp(reg, "EAX") == 0) return 0;
if (strcmp(reg, "ECX") == 0) return 1;
if (strcmp(reg, "EDX") == 0) return 2;
```

Prompt: En base al programa que me diste, dame una función que divida un entero de 32 bits en bytes.

Resultado:

```
/* ---- Funciones auxiliares ---- */

static void write_imm32(uint8_t* buffer, int32_t value)
{
    buffer[0] = value & 0xFF;
    buffer[1] = (value >> 8) & 0xFF;
    buffer[2] = (value >> 16) & 0xFF;
    buffer[3] = (value >> 24) & 0xFF;
}
```

Prompt: Crea la función para el ModRM.

Resultado:

```
static uint8_t make_modrm(uint8_t mod, uint8_t reg, uint8_t rm)
{
    return (mod << 6) | (reg << 3) | rm;
}
```

Prompt: Crea las instrucciones para el .h del encoder para conectarlo con el parser.

Resultado:

```
#include <stdint.h>
#include "parser.h"
```




```
uint32_t codificar_instruccion_ia32(  
    Instruction* inst,  
    uint8_t* output_buffer  
);
```

Integrante 5:

Tomando en cuenta el contenido del PDF del proyecto, ayúdame a trabajar la parte del apartado 13.5 (Integrante 5), que corresponde a Formato Objeto y Linker. Quiero que el código se adapte a las partes ya hechas del proyecto y que sea fácil de integrar con el trabajo de los demás integrantes, sin modificar ni romper las funciones que ya existen.

Primero revisa lo que se pide en el documento y después genera el código en C que incluya los puntos mencionados en ese apartado. También indica qué archivos nuevos tendría que crear dentro de las carpetas del proyecto y cómo se conectarán con el ensamblador que ya está desarrollado. Si es posible, incluye comentarios simples en el código para que mis compañeros puedan integrar más fácilmente y entender cómo funciona cada parte.

Código generado:

object_file.h:

```
#ifndef OBJECT_FILE_H  
#define OBJECT_FILE_H  
  
#include <stdint.h>  
  
/*  
 * FORMATO OBJETO PROPIO DEL PROYECTO  
 *  
 * Estructura física del archivo:  
 *  
 * [HEADER]  
 * [CODIGO MAQUINA]  
 * [TABLA DE SIMBOLOS]  
 * [TABLA DE RELOCACIONES]  
 *  
 * Este formato será utilizado por el Linker para  
 * combinar múltiples módulos ensamblados.  
 */  
  
#define OBJ_MAGIC 0x31424F4A  
  
/* Encabezado principal del archivo objeto */  
typedef struct {  
    uint32_t magic;  
    uint32_t code_size;
```



```
uint32_t symbol_count;
uint32_t relocation_count;
} ObjectHeader;

/*
 * Copia serializable de Symbol.
 *
 * Se mantiene independiente del backend para que
 * el archivo pueda ser leído posteriormente.
 */
typedef struct {
    char name[32];
    uint32_t address;
    uint8_t is_global;
    uint8_t is_extern;
} ObjectSymbol;

/*
 * Entrada de relocación.
 *
 * Indica al linker:
 * - qué símbolo buscar
 * - dónde aplicar el parche
 * - cuántos bytes modificar
 */
typedef struct {
    char symbol_name[32];
    uint32_t patch_pos;
    uint32_t size;
} RelocationEntry;

/* Escritura de archivo objeto */
int write_object_file(
    const char* filename,
    uint8_t* code,
    uint32_t code_size
);

/* Lectura de archivo objeto */
int read_object_file(
    const char* filename,
    uint8_t** code,
    uint32_t* code_size
);

/* Herramienta de depuración */
void dump_object_file(const char* filename);
```



#endif



object_file.c:

```
#include "../include/object_file.h"
#include "../include/backend.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * Esta función es llamada al terminar la
 * Segunda Pasada del ensamblador.
 *
 * Entradas:
 * - código máquina generado por el Integrante 4
 * - tabla de símbolos del Integrante 3
 * - tabla de fixups del Integrante 3
 *
 * Salida:
 * - archivo .obj listo para el linker
 */
int write_object_file(
    const char* filename,
    uint8_t* code,
    uint32_t code_size
)
{
    FILE* fp = fopen(filename,"wb");

    if(!fp)
    {
        printf("Error creando %s\n",filename);
        return 0;
    }

    ObjectHeader hdr;

    hdr.magic = OBJ_MAGIC;
    hdr.code_size = code_size;
    hdr.symbol_count = symbol_count;
    hdr.relocation_count = fixup_count;

    /* Escribir encabezado */
    fwrite(&hdr,sizeof(hdr),1,fp);
```



```
/* Escribir código máquina */
fwrite(code,1,code_size,fp);

/* Exportar tabla de símbolos */
for(int i=0;i<symbol_count;i++)
{
    ObjectSymbol sym;

    memset(&sym,0,sizeof(sym));

    strcpy(sym.name,symbol_table[i].name);

    sym.address =
        symbol_table[i].address;

    sym.is_global =
        symbol_table[i].is_global;

    sym.is_extern =
        symbol_table[i].is_extern;

    fwrite(&sym,sizeof(sym),1,fp);
}

/* Exportar tabla de relocalaciones */
for(int i=0;i<fixup_count;i++)
{
    RelocationEntry rel;

    memset(&rel,0,sizeof(rel));

    strcpy(
        rel.symbol_name,
        fixup_table[i].symbol_name
    );

    rel.patch_pos =
        fixup_table[i].patch_pos;

    rel.size =
        fixup_table[i].size;

    fwrite(&rel,sizeof(rel),1,fp);
}

fclose(fp);
```



```
printf(
    "[OBJ] Archivo objeto generado: %s\n",
    filename
);

return 1;
}

/*
 * Utilidad de depuración.
 *
 * Permite inspeccionar rápidamente un archivo
 * objeto sin necesidad de enlazarlo.
 */
void dump_object_file(const char* filename)
{
    FILE* fp = fopen(filename,"rb");

    if(!fp)
    {
        printf("No se pudo abrir %s\n",filename);
        return;
    }

    ObjectHeader hdr;

    fread(&hdr,sizeof(hdr),1,fp);

    printf("\n===== OBJECT DUMP =====\n");

    printf("Magic      : %X\n",hdr.magic);
    printf("Code Size   : %u\n",hdr.code_size);
    printf("Symbols     : %u\n",hdr.symbol_count);
    printf("Relocations : %u\n",hdr.relocation_count);

    fclose(fp);
}
```

linker.h:

```
#ifndef LINKER_H
#define LINKER_H

/*
 * Linker del proyecto IA-32.
 *
 * Responsabilidades:
```



- * - leer múltiples archivos objeto
- * - resolver símbolos globales
- * - resolver símbolos externos
- * - aplicar relocalaciones
- * - generar binario final
- */

```
int link_objects(  
    const char* output_file,  
    const char** object_files,  
    int object_count  
);
```

```
#endif
```

Linker.c:

```
#include "../include/linker.h"  
#include "../include/object_file.h"
```

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>
```

```
#define MAX_MODULES 16  
#define MAX_FINAL_BINARY 65536
```

```
/*  
 * Tabla global de símbolos del linker.  
 *  
 * Aquí se fusionarán los símbolos exportados  
 * por todos los módulos objeto.  
 */
```

```
typedef struct  
{  
    char name[32];  
    uint32_t address;  
} GlobalSymbol;
```

```
static GlobalSymbol global_table[512];  
static int global_count = 0;
```

```
/*  
 * Busca un símbolo dentro de la tabla global  
 * construida durante el enlace.
```





*/

UNAM

```
static int buscar_global(const char* name)
{
    for(int i=0;i<global_count;i++)
    {
        if(strcmp(global_table[i].name,name)==0)
            return i;
    }

    return -1;
}
```

/*

* Función principal del linker.

*

* Flujo:

*

* 1) Leer todos los objetos

* 2) Construir tabla global

* 3) Resolver símbolos externos

* 4) Aplicar relocalaciones

* 5) Generar binario final

*

* NOTA:

* Actualmente la implementación realiza

* la fusión básica de módulos.

*

* Las fases 2, 3 y 4 están preparadas para

* ampliarse cuando el Integrante 3 termine

* la lógica completa de símbolos externos.

*/

```
int link_objects(
    const char* output_file,
    const char** object_files,
    int object_count
)
{
    FILE* out;

    uint8_t final_binary[MAX_FINAL_BINARY];

    uint32_t final_size = 0;

    for(int m=0;m<object_count;m++)
    {
        FILE* fp = fopen(object_files[m],"rb");

        if(!fp)
```





```
{
    printf(
        "No se pudo abrir %s\n",
        object_files[m]
    );

    return 0;
}

ObjectHeader hdr;

fread(&hdr,sizeof(hdr),1,fp);

if(hdr.magic != OBJ_MAGIC)
{
    printf(
        "Formato objeto invalido\n"
    );

    fclose(fp);
    return 0;
}

/*
 * Copiar el código máquina del módulo
 * al binario final.
 */
fread(
    final_binary + final_size,
    1,
    hdr.code_size,
    fp
);

final_size += hdr.code_size;

fclose(fp);
}

out = fopen(output_file,"wb");

if(!out)
{
    printf(
        "No se pudo crear %s\n",
        output_file
    );
}
```




```
    return 0;
}

fwrite(
    final_binary,
    1,
    final_size,
    out
);

fclose(out);

printf(
    "[LINKER] Binario final generado: %s (%u bytes)\n",
    output_file,
    final_size
);

return 1;
}
```

